Mode-A CDCL Branching

First Policy-Level Test of MAAT v1.6-T Structural Search

Christof Krieg
MAAT Project – Independent Research
https://maat-research.com

2026

Abstract

MAAT v1.6 reframes structural selection as search geometry: structural supports are not only diagnostics of completed systems, but candidate coordinates that may guide search before full solving or inference is performed. The v1.6-T triage specification makes this claim testable by requiring policy-level benchmarks against domain heuristics.

This paper reports the first such benchmark. We implement the Mode-A acquisition rule

$$A(c) = \mathrm{prog}(c) - \tau\, h_{\mathrm{MAAT}}(c)$$

inside a small transparent CDCL solver and compare it against VSIDS, progress-only, MAAT-only, and random branching on 99 generated CNF instances from random, planted, modular, XOR, graph-coloring, and pigeonhole families. The predeclared settings are fixed seed 62062, conflict budget 3500, $\beta = 8.0$, $\tau = 0.38$, and rollout horizon $h = 0$.

The result is mixed and therefore informative. All policies solve all instances. Mode A ties VSIDS in median compute cost and obtains 44 wins, 47 losses, and 8 ties against VSIDS, but its mean regret is positive (+7.4619) and it does not beat the progress-only ablation globally. Thus the global $h = 0$ structural penalty does not yet earn compute as a universal CDCL branching term. At the same time, the family-level results show a structured positive signal: Mode A improves over VSIDS on modular SAT and remains competitive on several structured families, while it loses sharply on pigeonhole formulas.

The main conclusion is not that Mode A wins. It is that v1.6-T has now passed from score-level speculation to policy-level falsifiability. Structural search helps where exploitable structure is present, hurts where the formula is globally symmetric or structure-poor, and therefore motivates the next test: structure-gated Mode A.

1 Purpose

Previous SAT papers in the MAAT series tested whether structural diagnostics predict hardness. Paper 50 studied graph-local hardness features, Paper 50b showed that frustration tails carry information lost by scalar compression, Paper 55 validated defect-field features against CDCL statistics, Paper 57 decomposed SAT hardness into mean, tail, cluster, and scale projections, and Paper 59 tested MAAT v1.6 as an active DPLL branching heuristic.

Those tests did not yet answer the policy question posed by MAAT v1.6-T:

> *Does a structural search score reduce compute-to-solution when inserted into a conflict-learning solver?*

This paper is the first direct T1 benchmark. It embeds a Mode-A structural acquisition rule in a transparent CDCL solver and measures paired compute-to-solution regret against VSIDS. The goal is not to compete with industrial SAT solvers. The goal is to determine whether the structural term earns compute under a shared CDCL loop. The full v1.6-T triage specification is given in a companion framework note; the present paper instantiates only its first obligation, T1.

2 Mode A in MAAT v1.6-T

The v1.6-T triage specification distinguishes two usage modes. Mode F treats structural selection as a feasibility prior: it filters structurally broken candidates while a domain heuristic ranks the survivors. Mode A is stronger: structural support directly enters the acquisition rule.

The tested acquisition score is

$$\boxed{A(c) = \mathrm{prog}(c) - \tau\, h_{\mathrm{MAAT}}(c)} \tag{1}$$

where c is a candidate branching variable, $\mathrm{prog}(c)$ is a local CDCL progress score, $h_{\mathrm{MAAT}}(c)$ is a bounded structural risk score, and $\tau \geq 0$ controls the structural penalty. Candidate selection is made by a softmax policy,

$$\pi(c \mid n) \propto \exp[\beta A(c)], \tag{2}$$

with fixed $\beta = 8.0$. The rollout horizon is $h = 0$, so $h_{\mathrm{MAAT}}(c)$ is the immediate candidate score rather than a look-ahead cost-to-go.

The structural risk is computed from local support coordinates

$$(H, B, S, V, R_{\mathrm{rob}}),$$

with the v1.2.1 closure

$$R_{\mathrm{resp}} = (HBV)^{1/3}, \tag{3}$$

$$R_{\mathrm{rob}} = \min\Big(R_{\mathrm{resp}}, (HBSV)^{1/4}\Big). \tag{4}$$

Operationally, H and B measure polarity consistency and balance, S measures short-clause pressure, V measures active clause-variable connectivity, and R_{rob} bounds the combined support.

The benchmark tests four questions:

1. Does Mode A beat VSIDS in paired compute cost?
2. Does Mode A beat the progress-only ablation?
3. Does MAAT-only branching fail when goal progress is removed?
4. Are gains and losses uniform, or family-dependent?

3 Transparent CDCL Solver

The solver is deliberately small and auditable. It implements:

- Boolean assignments with decision levels,
- repeated unit propagation,
- conflict detection,
- first-UIP-style clause resolution when possible,
- learned clauses,
- VSIDS variable bumping and decay,
- phase memory,
- fixed conflict budget.

It does not implement restarts, LRB/CHB, watched literals, clause database reduction, inprocessing, preprocessing, restarts, or modern solver engineering. This limitation is intentional: every tested policy shares exactly the same propagation and learning machinery. Only the branching policy changes.

The tested policies are:

Policy	**Meaning**
VSIDS	standard activity-driven CDCL baseline.
progress-only	$\mathrm{prog}(c)$ without structural penalty.
MAAT-only	structural risk only, goal-blind.
Mode A	$\mathrm{prog}(c) - \tau h_{\mathrm{MAAT}}(c)$.
random	random unassigned variable.

4 Benchmark Design

The benchmark uses 99 generated CNF instances from six families:

random 3-SAT, planted 3-SAT, modular 3-SAT, XOR CNF, graph coloring, pigeonhole.

No external CNF dataset is redistributed. The seed, policy parameters, and conflict budget are fixed:

Parameter	**Value**
seed	62062
conflict budget	3500
β	8.0
τ	0.38
rollout horizon h	0
instances	99

The compute cost is

$$C_{\mathrm{compute}} = C + 0.30D + 0.004P + T\,B_{\mathrm{conf}}, \tag{5}$$

where C is the number of conflicts, D decisions, P propagations, T a timeout indicator, and B_{conf} the conflict budget. The main evaluation quantity is paired regret against VSIDS:

$$\Delta C_\pi = C_\pi - C_{\mathrm{VSIDS}}. \tag{6}$$

Negative regret means that policy π used less compute than VSIDS on the same instance.

5 Aggregate Results

All policies solve all 99 instances. This means the benchmark measures compute ordering rather than timeout separation. Table 1 reports aggregate policy statistics.

Policy	**Solved**	**Med. conflicts**	**Med. decisions**	**Med. cost**	**Mean cost**
Mode A	1.000	4.0	8.0	5.160	19.8564
Progress-only	1.000	3.0	7.0	5.160	13.6888
VSIDS	1.000	4.0	7.0	5.280	12.3945
MAAT-only	1.000	5.0	9.0	7.932	23.1309
Random	1.000	5.0	11.0	7.944	27.0231

Table 1: Aggregate CDCL policy statistics. Mode A ties progress-only in median cost and slightly improves over VSIDS in median cost, but it has higher mean cost than both.

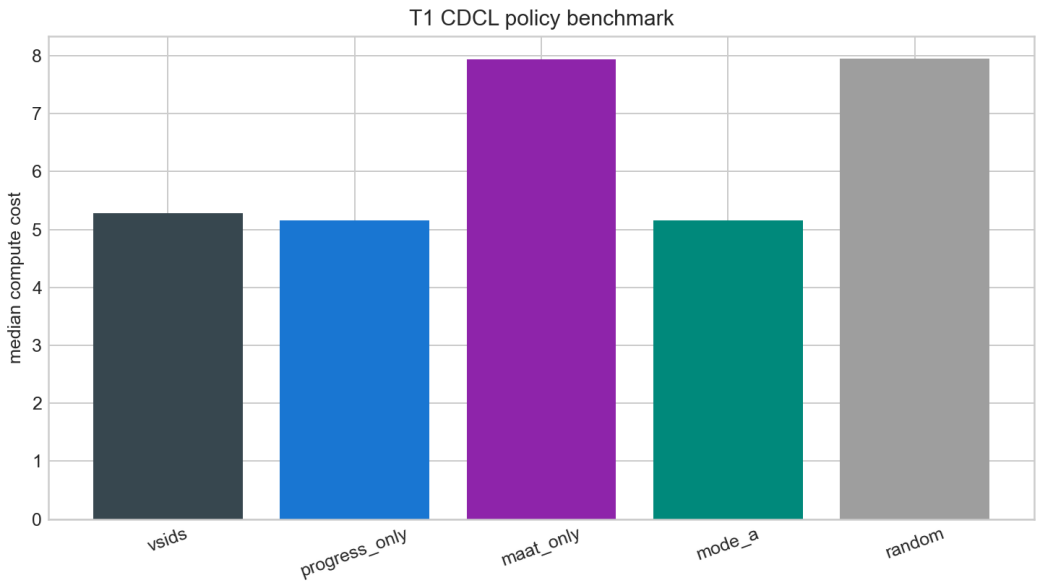

Figure 1: Median compute cost by branching policy. Mode A is competitive in median cost but does not dominate the progress-only or VSIDS baselines.

The paired regret table is more diagnostic.

Policy	**Mean regret**	**Med. regret**	**Mean ratio**	**Wins**	**Losses**
Progress-only	+1.2943	0.000	0.9845	31	24
MAAT-only	+10.7364	+0.576	1.6588	40	54
Mode A	+7.4619	0.000	1.2946	44	47
Random	+14.6286	+3.416	1.9130	27	72

Table 2: Paired compute-to-solution regret against VSIDS. Mode A has nearly balanced wins and losses but positive mean regret, indicating concentrated losses.

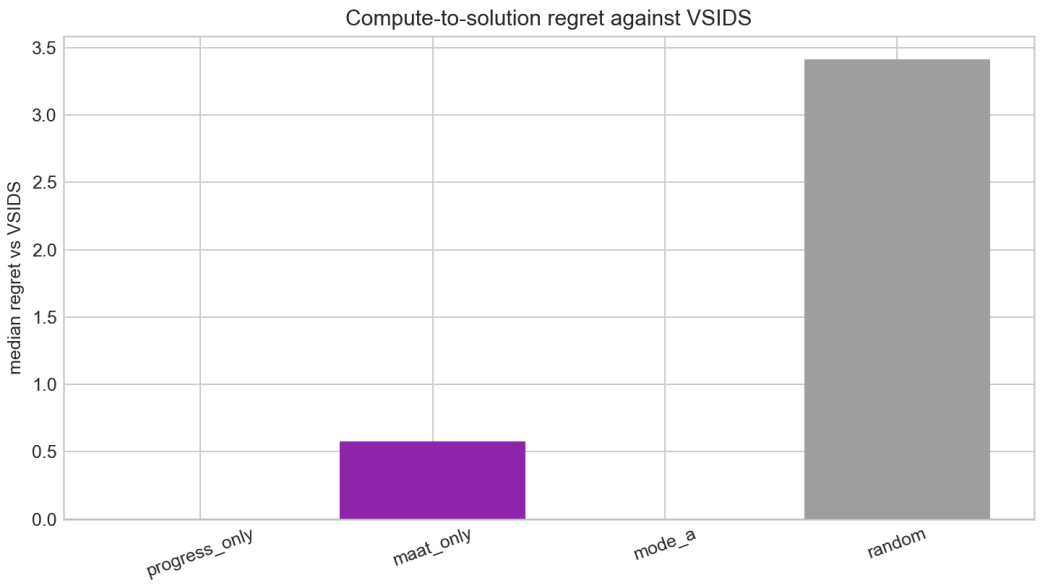

Figure 2: Median regret against VSIDS. Negative values would indicate less compute than VSIDS. The global Mode-A result is neutral in the median but positive in the mean.

6 Family-Level Structure

The aggregate result hides the main scientific signal. Mode A does not behave uniformly across formula families. Table 3 reports family-level median regret against VSIDS.

Family	**Progress-only**	**MAAT-only**	**Mode A**	**Random**
graph coloring	0.000	+0.148	0.000	+1.272
modular 3-SAT	0.000	+0.748	**-0.486**	+3.270
pigeonhole	+18.976	+69.624	+60.980	+72.520
planted 3-SAT	0.000	**-0.588**	**-0.428**	+0.596
random 3-SAT	0.000	+3.748	+0.776	+4.244
XOR CNF	0.000	+0.170	0.000	+3.556

Table 3: Family-level median regret against VSIDS. Negative regret means lower compute cost than VSIDS. Mode A is helpful on modular and planted instances but harmful on pigeonhole and random 3-SAT in this fixed $h = 0, \tau = 0.38$ setting.

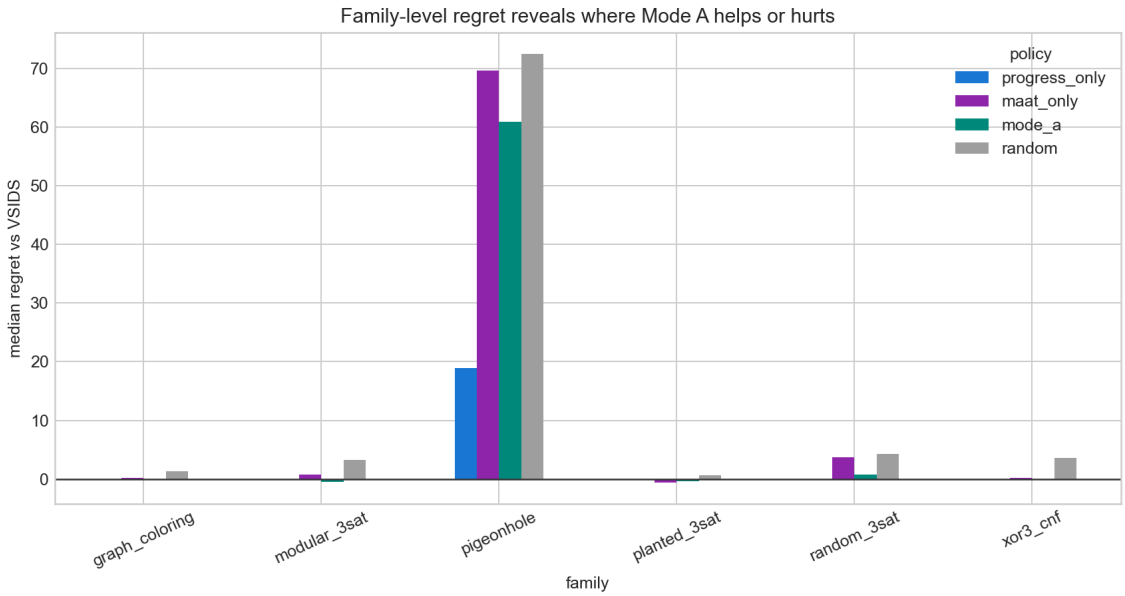

Figure 3: Family-level regret against VSIDS. The main result is not a uniform gain, but a structured family dependence.

This family dependence is the central finding. Modular formulas contain clustered variable-community structure. That is precisely the kind of low-frequency connectivity mode that previous MAAT transfer papers identified as partially portable. Pigeonhole formulas, by contrast, are globally symmetric and highly constrained but do not offer the same exploitable local community signal. In such cases the structural penalty can become pure branching drag.

7 Interpretation

The strongest conclusion is negative and should be stated plainly:

> *At $h = 0$, with fixed $\tau = 0.38$, global Mode A does not beat the progress-only ablation and therefore does not yet justify itself as a universal CDCL branching policy.*

This is not a failure of the test. It is the test doing its job. The v1.6-T specification required exactly this kind of policy-level falsifiability. The benchmark shows that a structural term must earn compute against the domain heuristic, not merely beat shuffled nulls or correlate with hardness.

The positive result is subtler. Mode A is not uniformly bad. It is family-sensitive. It improves median regret on modular SAT and planted SAT, ties several structured cases, and loses sharply on pigeonhole formulas. This supports the more precise interpretation:

structure helps where exploitable structure is present.

The regret distribution also has a MAAT-like shape. The median is neutral, but the mean is dragged upward by concentrated losses. The structural policy is itself tail-sensitive: most instances are harmless, while a few families impose large regret. This mirrors the earlier v1.4 lesson that global averages hide the decisive tail geometry.

8 Next Hypothesis: Structure-Gated Mode A

The natural next step is not simply to tune τ globally. The family-level result suggests a more structural policy:

$$\boxed{A_{\text{gated}}(c) = \text{prog}(c) - g(\mathcal{F})\,\tau\,h_{\text{MAAT}}(c)} \tag{7}$$

where $g(\mathcal{F}) \in [0,1]$ is an instance-level structure gate. For low-k, community-rich formulas such as modular SAT, $g(\mathcal{F})$ should be large. For globally symmetric or structure-poor formulas such as pigeonhole, $g(\mathcal{F})$ should be small and the solver should fall back toward VSIDS.

Candidate gates include:

- variance or concentration of the V-mode,
- modularity of the clause-variable graph,
- tail mass of local frustration fields,
- gap between progress-only and structure-only recommendations,
- early-run disagreement between VSIDS and Mode A.

This is triage of the triage layer. Instead of asking whether MAAT should always steer the solver, the next test asks whether MAAT can detect when structural steering is appropriate.

9 Limitations

- The solver is a transparent CDCL pilot, not an industrial solver. It lacks watched literals, restarts, LRB/CHB, clause database reduction, preprocessing, and modern inprocessing.
- All instances are synthetic and generated locally. No SAT Competition benchmark is used in this pilot.
- The conflict budget is low and all instances are solved, so the experiment measures compute ordering rather than hard timeout separation.
- The tested point $(\beta, \tau, h) = (8.0, 0.38, 0)$ is only one policy point. It falsifies the global $h = 0$ claim at that point, not all possible Mode-A variants. The tested point is only one point in a high-dimensional policy space; systematic hyperparameter sweeps are left for future work.
- The structural supports are heuristic SAT-local quantities. They are not derived from a complete theory of CDCL dynamics.

10 Reproducibility

The full experiment is included in the public repository:

https://github.com/Chris4081/structural-selection-principle

Experiment folder:

`experiments/sat_cdcl_mode_a_t1/`

Run:

```
cd experiments/sat_cdcl_mode_a_t1
python3 sat_cdcl_mode_a_t1.py
```

The script writes:

- `t1_cdcl_runs.csv`,
- `t1_cdcl_summary_by_policy.csv`,
- `t1_cdcl_regret_vs_vsids.csv`,
- `t1_cdcl_family_regret_vs_vsids.csv`,
- `t1_cdcl_summary.json`,
- three PNG figures.

Data attribution and license note. No external CNF dataset is redistributed. Instances are generated locally from standard synthetic SAT-family generators. CSV, JSON, and PNG files are derived reproducibility artifacts. Repository code is distributed under the MIT License. No endorsement by the authors of DPLL, CDCL, VSIDS, or any SAT-solver project is implied.

11 Conclusion

Paper 62 converts MAAT v1.6-T from a triage specification into a policy-level SAT benchmark. The result is scientifically useful because it is mixed. Mode A is not a universal improvement over VSIDS or progress-only branching at the tested $h = 0$ policy point. The structural penalty does not yet earn compute globally.

But the benchmark also reveals the right next direction. Mode A helps in structure-rich families such as modular SAT and hurts in families where structural steering is not useful. The next MAAT v1.6-T test should therefore be structure-gated Mode A: diagnose the formula first, activate structural branching only when the instance contains exploitable low-frequency structural modes, and fall back to VSIDS otherwise.

This is precisely the kind of result the framework needs: not a universal victory, but a falsifiable path from structural diagnostics to compute-aware search policy.

References

[1] M. Davis, G. Logemann, and D. Loveland, "A machine program for theorem-proving," *Communications of the ACM* **5**, 394–397 (1962).

[2] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A search algorithm for propositional satisfiability," *IEEE Transactions on Computers* **48**, 506–521 (1999).

[3] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, "Chaff: Engineering an efficient SAT solver," in *Proceedings of the 38th Design Automation Conference*, 530–535 (2001).

[4] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing*, SAT 2003, Lecture Notes in Computer Science **2919**, 502–518 (2004).

[5] A. Biere, M. Heule, H. van Maaren, and T. Walsh (eds.), *Handbook of Satisfiability*, IOS Press, Amsterdam (2009).

[6] C. Krieg, "Graph-Local Defects and Structural Hardness in SAT," preprint (2026).

[7] C. Krieg, "CDCL Hardness Prediction on Standard SAT Families," preprint (2026).

[8] C. Krieg, "MAAT v1.6 Guided SAT Search," preprint (2026).

[9] C. Krieg, "MAAT v1.6-T: The Triage Instantiation," framework specification (2026).